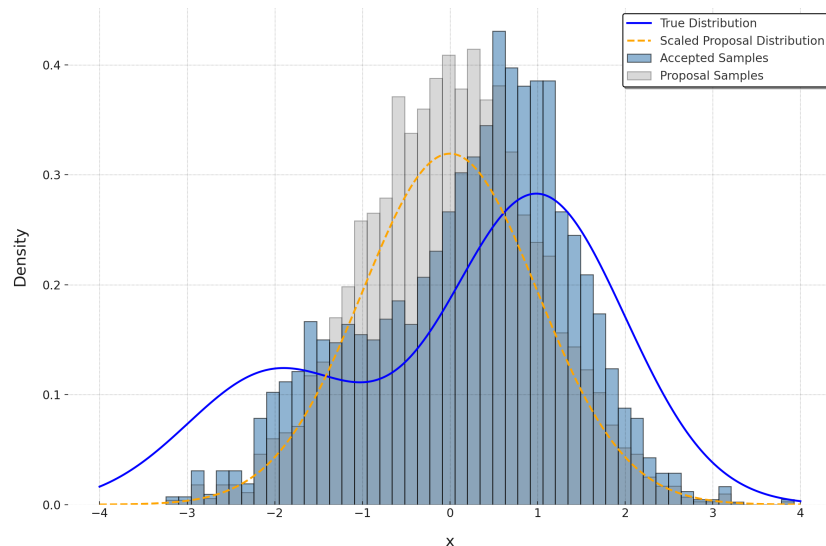


Rapport de projet

Améliorer les performances des GANs : Le Discriminator Rejection Sampling (DRS)

Julien Bernardi

Superviseur : Alexandre Vérine



*Ce rapport analyse les défis des GANs en termes de diversité et de qualité. Nous présentons des approches liées au **Discriminator Rejection Sampling (DRS)**, avec leurs résultats.*

Date : 3 janvier 2025

Table des matières

1. Introduction

- Problématique des GANs : diversités limitées et qualité inégale des échantillons.
- Introduction des approches : Discriminator Rejection Sampling (DRS) et Soft Truncation.

2. Présentation du DRS

- Définition du rejection sampling et intuition générale.
- Formulation mathématique

3. Implémentations des différentes méthodes

- **DRS Simpliste :**
 - Probabilité fondée sur le score du discriminateur $D(x)$
 - Définition d'un facteur M global
- **DRS Avancé :**
 - Introduction du ratio amélioré $\frac{D(x)}{1-D(x)}$.
 - Comparaison avec le DRS simpliste.
- **DRS avec Soft Truncation :**
 - Limitation des valeurs extrêmes avec $z \times scale$.
 - Amélioration des scores du discriminateur.
- **DRS avec Soft Truncation Adapté :**
 - Utilisation dynamique des scores du discriminateur.
 - Réglages et adaptabilité selon les échantillons.

4. Conclusion

- Synthèse des résultats obtenus.
- Perspectives pour des GANs plus robustes et polyvalents.

1 Introduction

Les réseaux antagonistes génératifs (GANs) sont devenus une approche incontournable pour la génération d'images, de textes et d'autres données synthétiques. Cependant, ces modèles rencontrent deux problèmes majeurs :

- **Mode collapse** : Une situation où le générateur se concentre sur un sous-ensemble limité des modes de la distribution cible, conduisant à une diversité insuffisante des échantillons générés.
- **Qualité inférieure des échantillons** : Certains échantillons présentent des artefacts ou ne respectent pas fidèlement la distribution cible.

Pour résoudre ces problèmes, des méthodes avancées comme le *Discriminator Rejection Sampling (DRS)* et la *Soft Truncation* ont été introduites. Ces approches permettent d'améliorer la qualité et la diversité des échantillons en exploitant les scores du discriminateur pour filtrer les échantillons produits par le générateur. Ce sont principalement sur ces méthodes que va s'appuyer notre projet.

La section suivante introduit les fondements théoriques du DRS, accompagnés des formulations mathématiques nécessaires à sa compréhension. Nous rappelons ici que ce projet est réalisé sur le dataset MNIST, avec exactement le même générateur, et un discriminateur ne contenant que 3 couches linéaires contrairement à celui fourni qui en contenait 4. En effet, les résultats ont montré qu'un discriminateur moins bon était plus approprié dans le cadre du DRS, notamment en raison de sa précision parfois trop stricte.

2 Présentation du Discriminator Rejection Sampling (DRS)

Le *Discriminator Rejection Sampling (DRS)* est une technique qui exploite les scores du discriminateur pour rejeter les échantillons de faible qualité, améliorant ainsi la conformité à la distribution cible. Lorsque le discriminateur est entraîné de manière optimale, il converge vers :

$$D^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_g(x)}, \quad (1)$$

où :

- $p_{data}(x)$ est la distribution réelle des données.
- $p_g(x)$ est la distribution des données générées par le générateur.

Ce ratio optimal, $D^*(x)$, constitue la base théorique qui guidera notre implémentation du *Discriminator Rejection Sampling (DRS)* tout au long de ce projet. Voici une présentation détaillée de son fonctionnement théorique.

2.1 Intuition Générale

Le DRS repose sur le principe de *rejection sampling*, une méthode statistique permettant de générer des échantillons suivant une distribution cible $p_{data}(x)$, en utilisant une distribution auxiliaire $p_g(x)$. Dans notre contexte des GANs, le générateur fournit $p_g(x)$, tandis que le discriminateur est utilisé pour estimer la vraisemblance d'un échantillon x par rapport à la distribution cible $p_{data}(x)$.

2.2 Formulation Mathématique

Pour chaque échantillon x généré par le générateur, le score du discriminateur $D(x)$ est interprété comme une estimation de la probabilité que x appartienne à la distribution des données réelles.

Probabilité d’acceptation : La probabilité d’accepter un échantillon est donnée par :

$$P_{accept}(x) = \frac{D(x)}{M}, \quad (2)$$

où :

- $D(x)$ est la sortie du discriminateur pour l’échantillon x .
- $M = \max(D(x))$ est une constante de normalisation assurant que $P_{accept}(x) \leq 1$.

Rejet des échantillons : Un échantillon x est accepté avec la probabilité $P_{accept}(x)$, et rejeté autrement. Cette méthode favorise les échantillons ayant des scores $D(x)$ élevés, réduisant ainsi les artefacts/échantillons de mauvaise qualité par rapport aux données réelles.

3 Implémentations des Différentes Méthodes

Cette section décrit les quatre méthodes principales utilisées pour implémenter le *Discriminator Rejection Sampling (DRS)* et leurs variantes. Chaque méthode est détaillée pour en expliquer le fonctionnement, suivie de l’extrait de code correspondant et d’une référence aux analyses des résultats en annexe.

3.1 DRS Simpliste

Le **DRS Simpliste** repose sur l’utilisation directe des scores du discriminateur pour accepter ou rejeter des échantillons.

Méthode :

- Calculer le score du discriminateur $D(x)$ pour chaque échantillon.
- Normaliser les scores avec un facteur global M , tel que $M = \max(D(x))$ comme étant le maximum des maximums des scores calculés sur plusieurs batches (mis à jour si le max des scores d’un batch lui est supérieur). En réalité ici, comme le score $D(x)$ est déjà compris entre 0 et 1 (la sortie étant une sigmoïde), définir M n’est qu’optionnel mais nous préférons le faire ici pour faciliter la compréhension pour la suite.
- Accepter un échantillon avec une probabilité $P_{accept}(x) = \frac{D(x)}{M}$.

Modifications apportées lors de la génération des samples : DRS Simpliste

```
D_scores = discriminator(x.view(args.batch_size, -1)).squeeze().detach().cpu()
batch_max = D_scores.max().item()
M_global_max = max(M_global_max, batch_max)
P_accept = D_scores / M_global_max
for k in range(x.shape[0]):
    if n_samples_drs < args.num_samples:
        u = torch.rand(1).item()
        if u < P_accept[k]:
```

Analyse des Résultats ([Annexe A](#)) : Les samples acceptés sont encore parfois de basse qualité, on rejette également de bons samples. La distinction entre samples de bonne et de mauvaise qualité est donc encore assez floue. De plus, les diversités inter-classe et intra-classe restent limitées ([Annexe A](#)).

Précision : 0.48 - 0.50, Recall : 0.21

3.2 DRS Avancé

Le **DRS Avancé** améliore le **DRS Simpliste** en utilisant un ratio beaucoup plus approprié toujours lié au score du discriminateur, ainsi qu'un calcul local de M pour chaque batch (le max des scores utilisé correspond toujours au maximum des scores du batch concerné). Comme introduit précédemment, l'entraînement du discriminateur optimal converge vers :

$$D^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_g(x)},$$

En utilisant cette relation, nous pouvons reformuler un nouveau ratio afin de mieux exploiter l'information entre les distributions. En effet, en utilisant $\frac{D^*(x)}{1-D^*(x)}$, nous obtenons un ratio révisé :

$$\frac{D^*(x)}{1 - D^*(x)} = \frac{p_{data}(x)}{p_g(x)}.$$

Pourquoi cette approche est-elle meilleure ?

- Le ratio $\frac{p_{data}(x)}{p_g(x)}$ met davantage l'accent sur la séparation entre les deux distributions, en isolant la contribution relative de chaque distribution.
- Cela permet un filtrage plus précis des échantillons générés, favorisant ceux qui appartiennent avec plus de certitude à la distribution réelle.

Méthode :

- Calculer la probabilité d'acceptation comme $P_{accept}(x) = \frac{D(x)}{1-D(x)+\varepsilon}$, où ε est un terme que l'on peut utiliser pour régulariser la probabilité, ici $\varepsilon = 10^{-6}$
- Recalculer M comme étant le maximum du ratio calculé localement sur chaque batch

Modifications apportées lors de la génération des samples : DRS Avancé

```
D_scores = discriminator(x_flattened).squeeze().detach().cpu()
ratios = D_scores / (1 - D_scores + 1e-6)
M_local = ratios.max().item()
P_accept = ratios / M_local
```

Analyse des Résultats ([Annexe B](#)) : La qualité et la diversité sont améliorées. En recalculant M localement, on s'assure que les échantillons rares (scores élevés dans leur batch respectif) ont une probabilité raisonnable d'être acceptés. Également le nouveau ratio favorise dorénavant plus les échantillons qui trompent efficacement le discriminateur. Précision : 0.51 - 0.52, Recall : 0.22

3.3 DRS avec Soft Truncation

Les échantillons obtenus n'étant toujours pas de qualité optimal, nous avons également implémenté une **Soft Truncation**. Cette méthode limite les valeurs extrêmes des vecteurs latents z avant de les envoyer au générateur, améliorant ainsi la qualité des échantillons.

Méthode :

- Appliquer $z = z \times 0.7$ avant de le passer au générateur lors de la génération et de l'entraînement.
- Appliquer le DRS comme dans la Section 3.2.

Modifications lors de la génération/training des samples : Soft Truncation

```
def soft_truncation(z, scale):  
    return z * scale  
  
z = torch.randn(args.batch_size, 100).to(device)  
z = soft_truncation(z, args.truncation_scale)
```

Analyse des Résultats ([Annexe C](#)) : L'utilisation d'une soft truncation produit comme attendu, une nette amélioration de la qualité des échantillons. Néanmoins, en perdant certaines données lors de la troncature, on perd énormément en diversité inter et intra-classe. On pourrait également jouer avec le facteur de troncature pour favoriser ou non la diversité.

Précision : 0.55, Recall : 0.16

3.4 DRS avec Soft Truncation Adapté

Nous venons de le voir, en appliquant la troncature et le DRS, il y a un arbitrage entre diversité et qualité. L'objectif de cette dernière méthode est de pouvoir obtenir le meilleur arbitrage possible, ie améliorer la qualité des échantillons tout en gardant une bonne diversité.

Méthode :

- Calculer *scale* dynamiquement en fonction de $D(x)$. L'objectif est de ne pas modifier les vecteurs latents z qui génèrent des échantillons déjà crédibles et de tronquer ceux qui en génèrent des peu crédibles.
- $scale = \frac{D(x)}{1-D(x)+\varepsilon}$, avec $\varepsilon > 0$ comme constante de régularisation. Le facteur *scale* est d'autant plus faible (et donc la troncature d'autant plus sévère) que l'échantillon est considéré comme peu crédible, et inversement. On clamp ensuite les valeurs pour éviter des modifications trop extrêmes.
- Appliquer le DRS comme dans la section 3.2.

Modifications lors de la génération/training des samples : Adaptive Truncation

```
def dynamic_soft_truncation(z, D_scores):  
    scaling_factors = D_scores / (1 - D_scores + 1e-6)  
    scaling_factors = torch.clamp(scaling_factors, 0.5, 1.2)  
    return z * scaling_factors.unsqueeze(1)  
  
z = torch.randn(args.batch_size, 100).to(device)  
D_scores = discriminator(generator(z).view(args.batch_size, -1)).squeeze().detach()  
z = dynamic_soft_truncation(z, D_scores)
```

Analyse des Résultats ([Annexe D](#)) : En ne modifiant pas/très peu les échantillons déjà crédibles, et en tronquant les vecteurs latents générant des samples peu crédibles, nous avons grandement amélioré la diversité tout en gardant une netteté plus qu'acceptable.

Précision : 0.53, Recall : 0.24

4 Conclusion

Dans ce rapport, nous avons exploré et implémenté diverses variantes du *Discriminator Rejection Sampling (DRS)* pour améliorer la qualité et la diversité des échantillons générés par les GANs. Les différentes méthodes, allant du DRS Simpliste à la Soft Truncation Adaptée, ont permis de mettre en évidence les avantages et les limites de chaque approche.

Malgré les améliorations apportées par les différentes variantes du *Discriminator Rejection Sampling (DRS)* décrites dans ce rapport, nos méthodes présentent plusieurs limitations importantes. L'efficacité de nos approches repose sur l'équilibre entre la qualité du générateur et celle du discriminateur. Nos méthodes sont cependant limitées par la nature des modèles utilisés, restreints à des couches linéaires. Cette contrainte réduit leur capacité à capturer des représentations complexes, impactant ainsi leur efficacité globale.

Un autre point critique est le compromis constant entre qualité et diversité des échantillons. Si certaines méthodes, comme le DRS Avancé ou la Soft Truncation, améliorent considérablement la qualité des échantillons, elles le font souvent au détriment de la diversité intra- et inter-classe. Cette perte de diversité peut être préjudiciable dans des contextes où la variété des échantillons générés est primordiale.

En conclusion, bien que nos méthodes basées sur le DRS aient montré des améliorations significatives dans certains aspects, elles restent perfectibles. S'inspirer d'algorithmes robustes comme Metropolis-Hastings pour mieux approcher la distribution cible ou explorer des variantes plus flexibles du DRS constitue des pistes prometteuses pour surmonter leurs limites et ouvrir de nouvelles perspectives dans la génération de données synthétiques.

Références :

- Goodfellow, I., et al. (2014). Generative Adversarial Networks.
- Azadi, S., Olsson, C., Darrell, T., Goodfellow, I., & Odena, A. (2019). Discriminator Rejection Sampling.
- Hastings, W. K. (1970). Monte Carlo Sampling Methods Using Markov Chains and Their Applications.
- Salimans, T., et al. (2016). Improved Techniques for Training GANs. *Advances in Neural Information Processing Systems (NeurIPS)*.

Annexe A : Résultats pour le DRS Simpliste

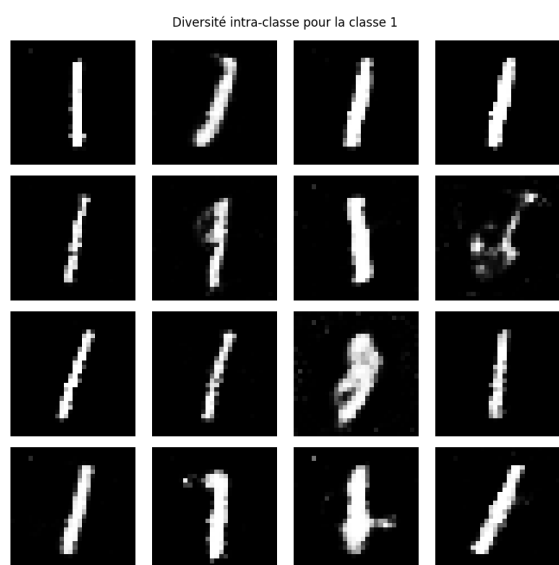


Figure 1A : Diversité intra-classe pour le DRS Simpliste

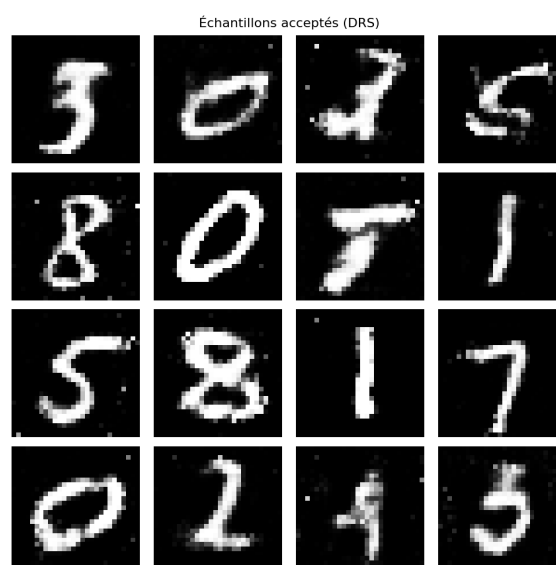


Figure 2A : Échantillons acceptés (DRS Simpliste)

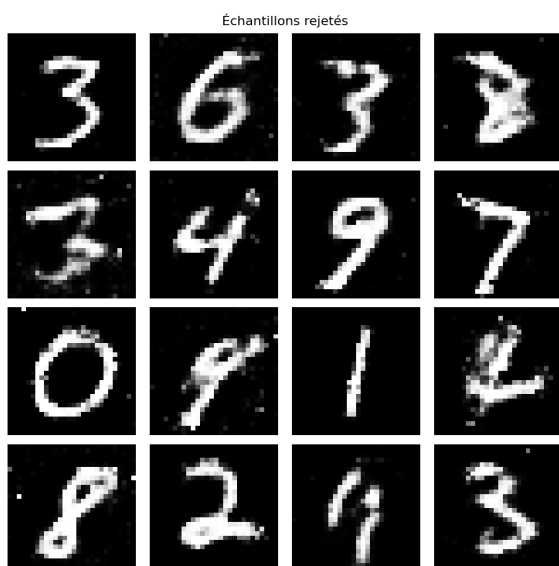


Figure 3A : Échantillons rejetés (DRS Simpliste)

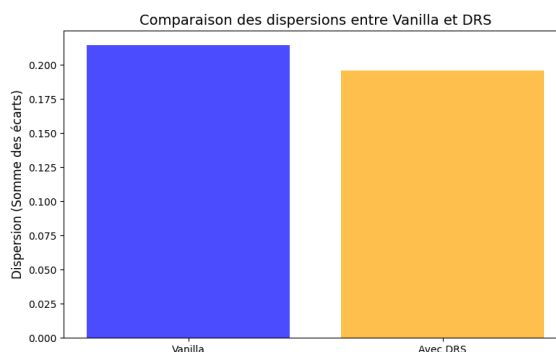


Figure 4A : Comparaison des dispersions entre Vanilla et DRS Simpliste

* Le graphique 4A représente la somme des écarts entre les proportions idéales des classes (10% pour chaque classe) et les proportions obtenues. Plus la somme est faible, plus c'est équilibré

Annexe B : Résultats pour le DRS Avancé

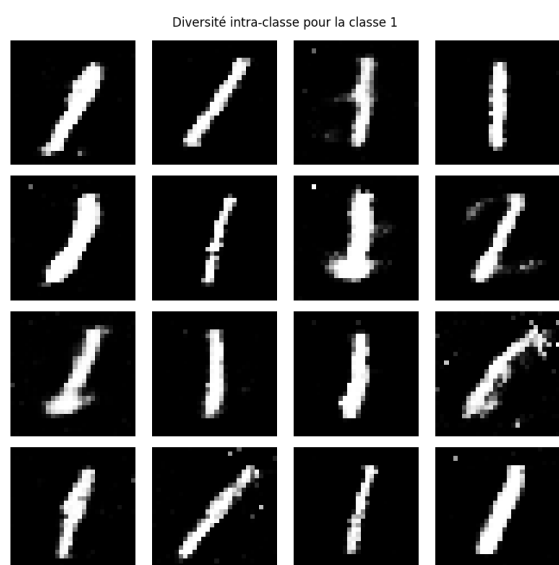


Figure 1B : Diversité intra-classe pour le DRS Avancé

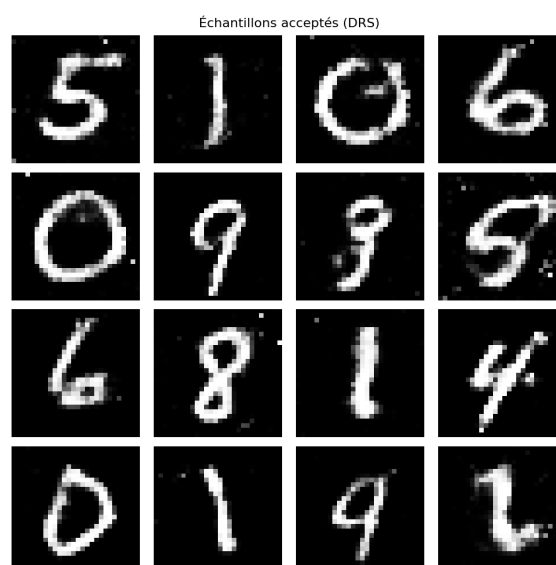


Figure 2B : Échantillons acceptés (DRS Avancé)

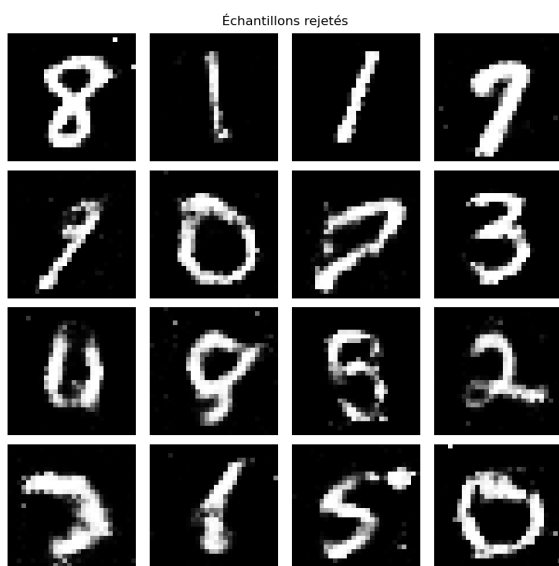


Figure 3B : Échantillons rejetés (DRS Avancé)

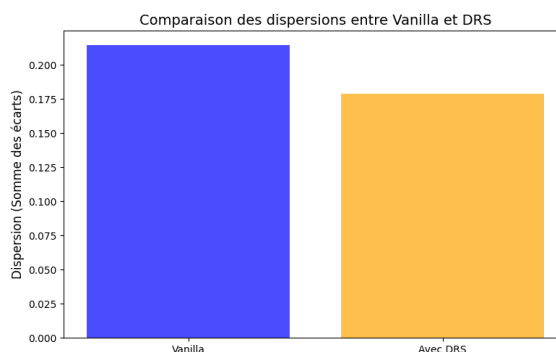


Figure 4B : Comparaison des dispersions entre Vanilla et DRS Avancé

Annexe C : Résultats pour le DRS avec Soft Truncation

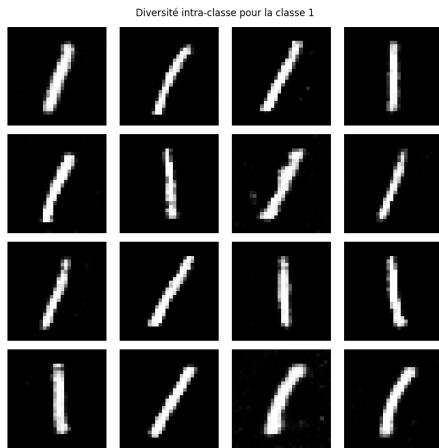


Figure 1C : Diversité intra-classe pour le DRS avec Soft Truncation

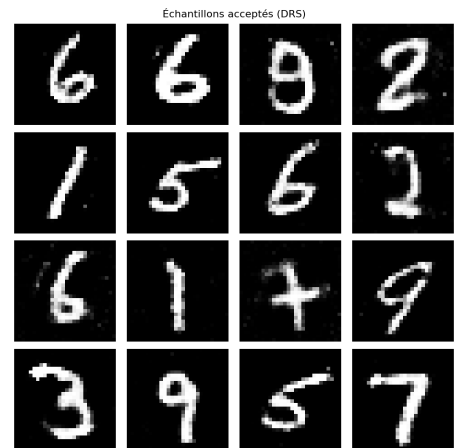


Figure 2C : Échantillons acceptés (DRS avec Soft Truncation)

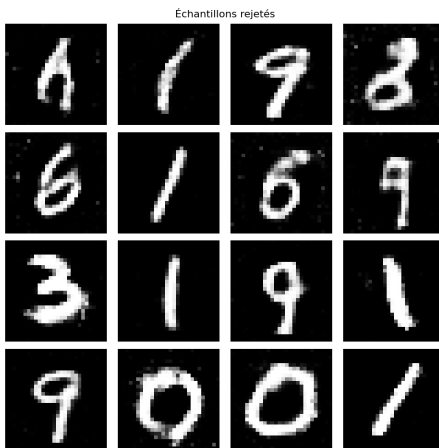


Figure 3C : Échantillons rejetés (DRS avec Soft Truncation)

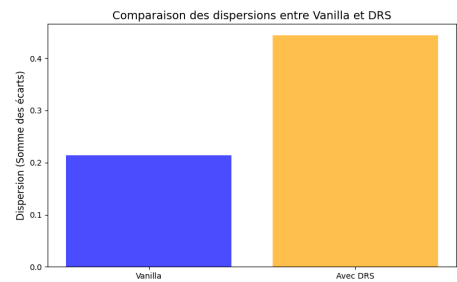


Figure 4C : Comparaison des dispersions entre Vanilla et DRS avec Soft Truncation

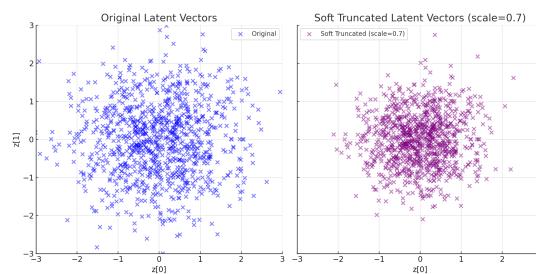


Figure 5C : Impact de la Soft Truncation sur les vecteurs latents

Annexe D : Résultats pour le DRS avec Soft Truncation Adaptée

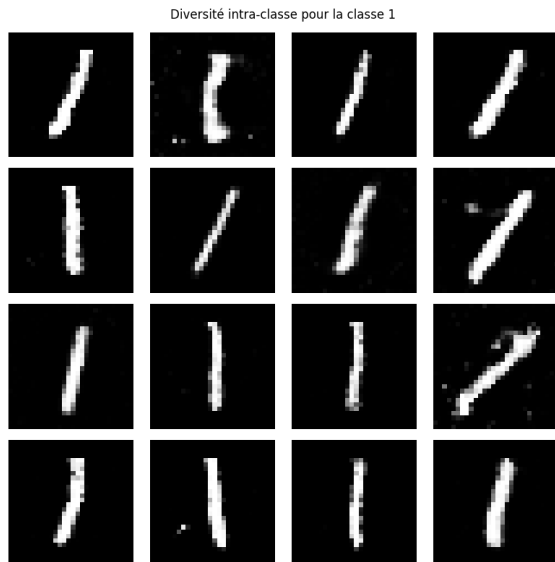


Figure 1D : Diversité intra-classe pour le DRS avec Soft Truncation Adaptée

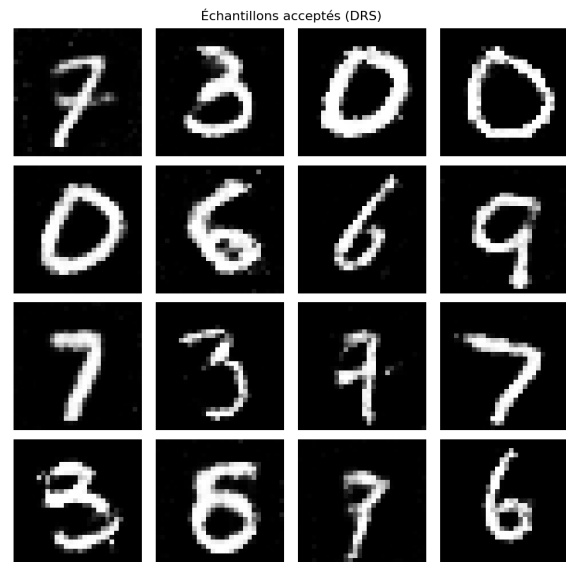


Figure 2D : Échantillons acceptés (DRS avec Soft Truncation Adaptée)

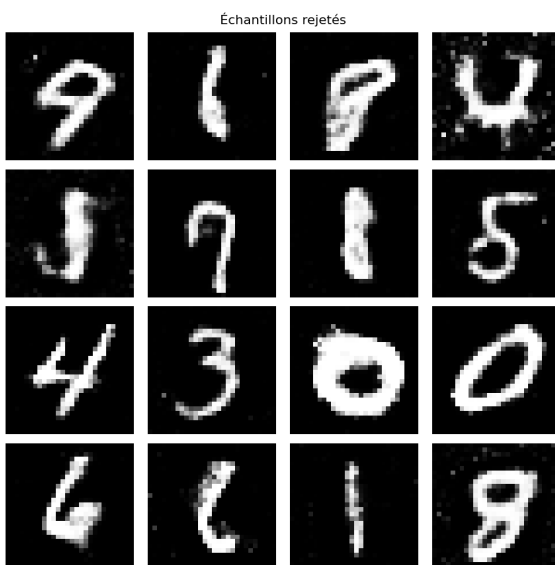


Figure 3D : Échantillons rejetés (DRS avec Soft Truncation Adaptée)

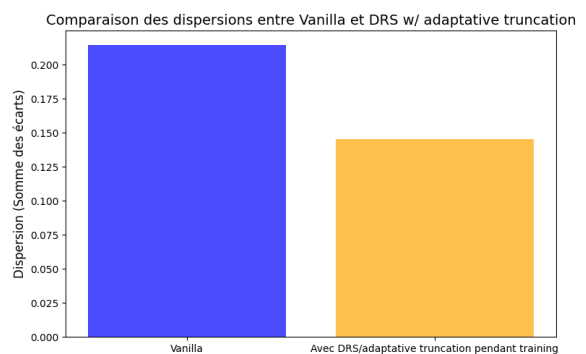


Figure 4D : Comparaison des dispersions entre Vanilla et DRS avec Soft Truncation Adaptée